Man-in-the-Browser Attacks

Last updated on March 5, 2026

By: Shiran Grinberg

How hackers use browser vulnerabilities to invisibly manipulate secure connections

Introduction

In our last article, we've demonstrated how attackers may evade detection when establishing C&C and data exfiltration channels by using *ICMP Tunneling* to obfuscate traffic.

But how is data collected in the first place? Naturally *Data Collection* techniques (MITRE TA0009: *Collection*[1]) are as diverse and abundant as they come, but some common examples may be:

- File System Access – Collection of data and information stored as static files on the victim's machine. This may include user-created files (Office and PDF files, photos, videos etc.) or files generated by applications or the OS (web browser bookmarks and history, registry values etc.).
- Queried Data – Collection of data through application and OS APIs and commands, instead of from the file system. For example, clipboard data can be gathered using various tools that leverage OS APIs[2].
- I/O Device Interception – May include victim machine's keyboard and mouse (see MITRE T1056: *Input Capture*[3]), screen (MITRE T1113: *Screen Capture*[4]), and microphone and webcam input (MITRE T1125: *Video Capture*[5]).

In this article, we'll show one common technique adversaries use to collect user data and even manipulate it to their advantage. In a nutshell, it's a form of a Man-in-the-Middle attack that focuses on web browsers as the attack vector.

The rest of the introduction discusses Man-in-the-Middle attacks and web browsers.

This is part of an extensive series of guides about Network Attacks

Man-in-the-Middle Attacks

When employing a man-in-the-middle attack, adversaries position themselves between two machines or network devices, in such a way that allows them to intercept and manipulate the traffic between both parties.

Usually this will be achieved by spoofing and relaying techniques: the adversary will falsely identify as an existing network host or service, tricking clients into accessing his machine instead of the actual service. Then, all requests from the client to the attacker will be relayed to the actual service, and all responses from the service will be relayed back to the client. The attacker now has free reign to intercept and manipulate the data-stream to his advantage.

Web Browsers and Browser Security

As we all know, web browsers are software application used to access and browse the World Wide Web. They are used on many different devices, and had a clientele of 4.3 billion people in 2019, according to *Internet World Stats*[6].

As technology advanced and the world wide web gained prominence, more and more information became accessible through web browsers. Furthermore, larger amounts of personal, exploitable data were now accessible online, as services migrated to web-based environments. From email clients and cloud storage services to banking and government – clients can now use their web browsers to wire funds, manage financial assets, receive and send emails, access personal medical information and much more.

Moreover, a similar process is noticeable in Enterprise environments – as many intranet services, such as knowledge management systems, file servers, DBs etc. now offer web interfaces for users.

Accordingly, new security measures were implemented to protect personal data. Today, it is not uncommon to see websites that support HTTPS, security token mechanisms, and multi-factor and out-of-band authentication. Additionally, an ever-growing arsenal of technologies are available to protect the services themselves from malware, injections, cross-site scripting (XSS) and more.

Essentially, nowadays copious amounts of exploitable data are available through the world wide web. Yet collecting it, either by interception or false authentication, is harder than ever due to advancing security solutions. Queue Man-in-the-Browser.

Man-in-the-Browser

In a Man-in-the-Browser attack (MITRE T1185: *Man-in-the-Browser*[7]), adversaries will exploit vulnerabilities in a victim's web browser to gain partial or full control over it. Controlling the browser, the attacker is now a man-in-the-middle between the graphical content shown to the victim and the requested servers – and can use this position to intercept and manipulate both parties as he wishes.

However – and here is the genius of this technique – as this is a client-side MITM attack, it is very hard to distinguish legitimate client behavior from the attacker's injected malicious activity. Essentially, an adversary may "inherit" the victim's *security context*, rendering himself almost invisible.

Consider the following example: a victim is trying to transfer funds online. He makes sure the website is legitimate and that the connection is secure, authenticates using his username, password and ID number, fills a CAPTCHA form, and even verifies the login attempt on his mobile device, as the website implements out-of-band authentication. At last, he fills in the relevant form and hits *Enter*. The web browser will now render the user's actions into an HTTP POST request to the server, including the input form data. Except, maliciously controlled by an adversary, the browser sends two POST requests – the original, and one designed to transfer funds to himself.

Man-in-the-Browser malware usually serve as trojan horses that run inside a specific process. In most modern web browsers, that means a specific tab (as each tab runs on a separate process). As the victim browses the internet, the malware will be *hooked* to these tabs. Three common attack vectors are:

- Compromised Browser Extensions – Browser extensions are downloadable modules that extend a browser's usability, and for this purpose are usually granted some level of control over the browser. For example, extensions may read cookies and other data, manipulate DOM elements and make POST and GET requests. Most modern extensions are usually just source code, and are therefore prone to malicious code injection.
- API Hooking – Browsers rely on OS APIs and DLLs to function properly. For example, web browser for Windows may rely on the *wininet* DLL to send and receive HTTP requests[8]. Malwares can alter the API flow in a way that allows them to act as a "man-in-the-middle" between the API and the process requesting it, granting them extensive control over the processes' functionality.
- Malicious Content Scripts – Content scripts are script files that run in the context of a web page on a client's browser. These scripts are able to read and manipulate web pages by using the Document Object Model (DOM). Malicious content scripts can be injected into web pages either by infecting the web page on the hosting server, or by infecting the HTTP response sent to a specific client using MITM techniques.

Tips From Expert

In my experience, here are tips that can help you better defend against data collection and Man-in-the-Browser (MITB) attacks:

1. Deploy Browser Isolation Solutions: This technology provides a robust defense by isolating web sessions and preventing direct interaction between malicious code and the local endpoint.
2. Implement Certificate Pinning: By ensuring that browsers only trust specific certificates for critical sites, you significantly reduce the risk of MITB attacks that rely on spoofed certificates.
3. Integrate Content Security Policy (CSP) Headers: CSP is a powerful tool that can restrict the types of scripts that can run in your web applications, making it difficult for attackers to inject malicious code.
4. Enable Continuous Session Monitoring: Keeping a close eye on user sessions for suspicious activity can help detect MITB attacks in real time.
5. Segment Sensitive Web Applications: Isolating critical applications from general browsing reduces the risk of exposure to potential threats and helps contain the damage in case of a breach.

Aviad Hasnis is the Chief Technology Officer at Cynet.
He brings a strong background in developing cutting edge technologies that have had a major impact on the security of the State of Israel. At Cynet, Aviad continues to lead extensive cybersecurity research projects and drive innovation forward.

Demonstration

In this demonstration we will use BeEF (Browser Exploitation Framework), an open-source pen-testing tool. It is somewhat similar to Metasploit, in that it is a modular exploit framework – but for web browsers.

BeEF hooks on to web browser instances, enabling adversaries to launch its expansive set of command modules inside a browser's security context. In addition, it boasts a graphical user interface that streamlines the attack process significantly.

The server runs on a Linux machine but clients are not really OS-dependent. That is because BeEF relies on JavaScript code injection as its initial hook.

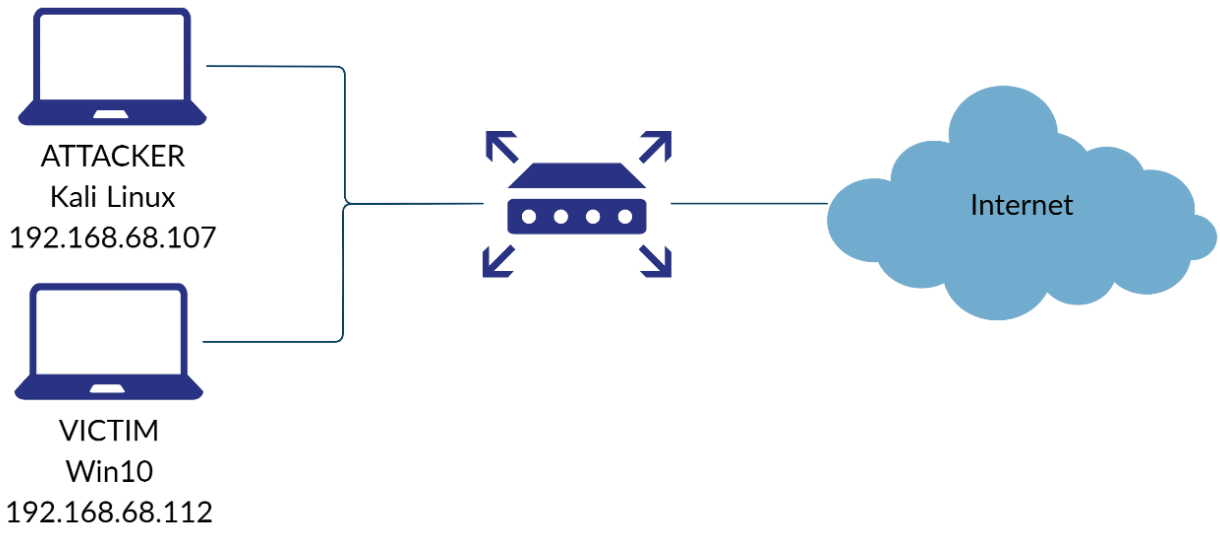

Figure 1: Demonstration network diagram

Step 1 – Initial Setup

First, the BeEF server should be initiated on the attacking machine. No additional parameters are mandatory:

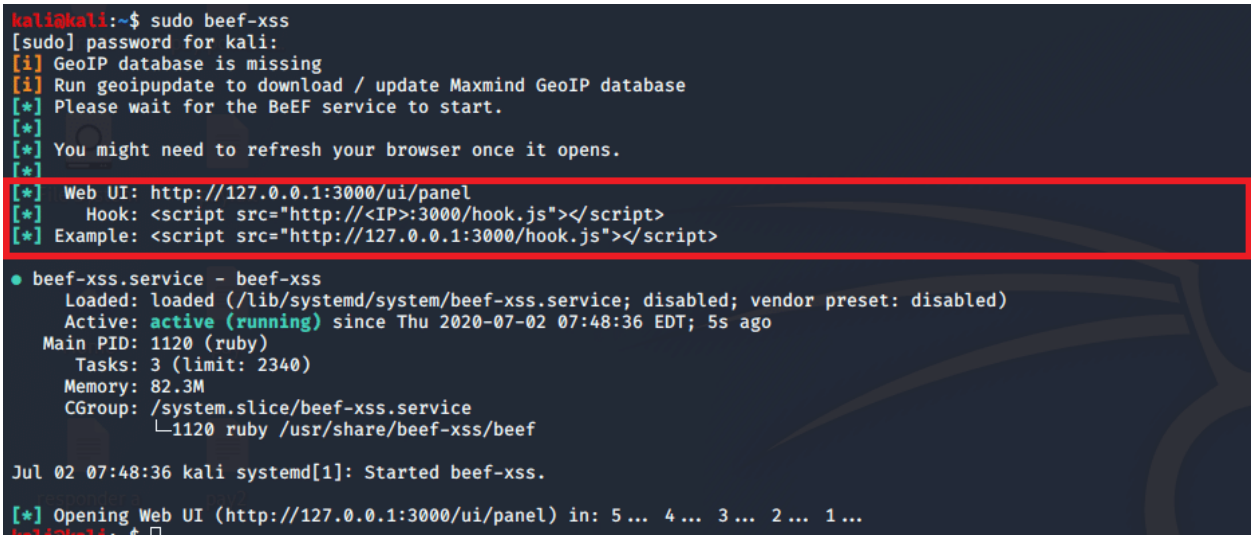

Figure 2: Starting the BeEF server. Notice the web UI and JS hook, marked in red.

The framework's Web UI is now available at *http://127.0.0.1:3000/ui/panel*, as highlighted above. The location of the malicious JavaScript code is also indicated. Here's the UI's authentication page.

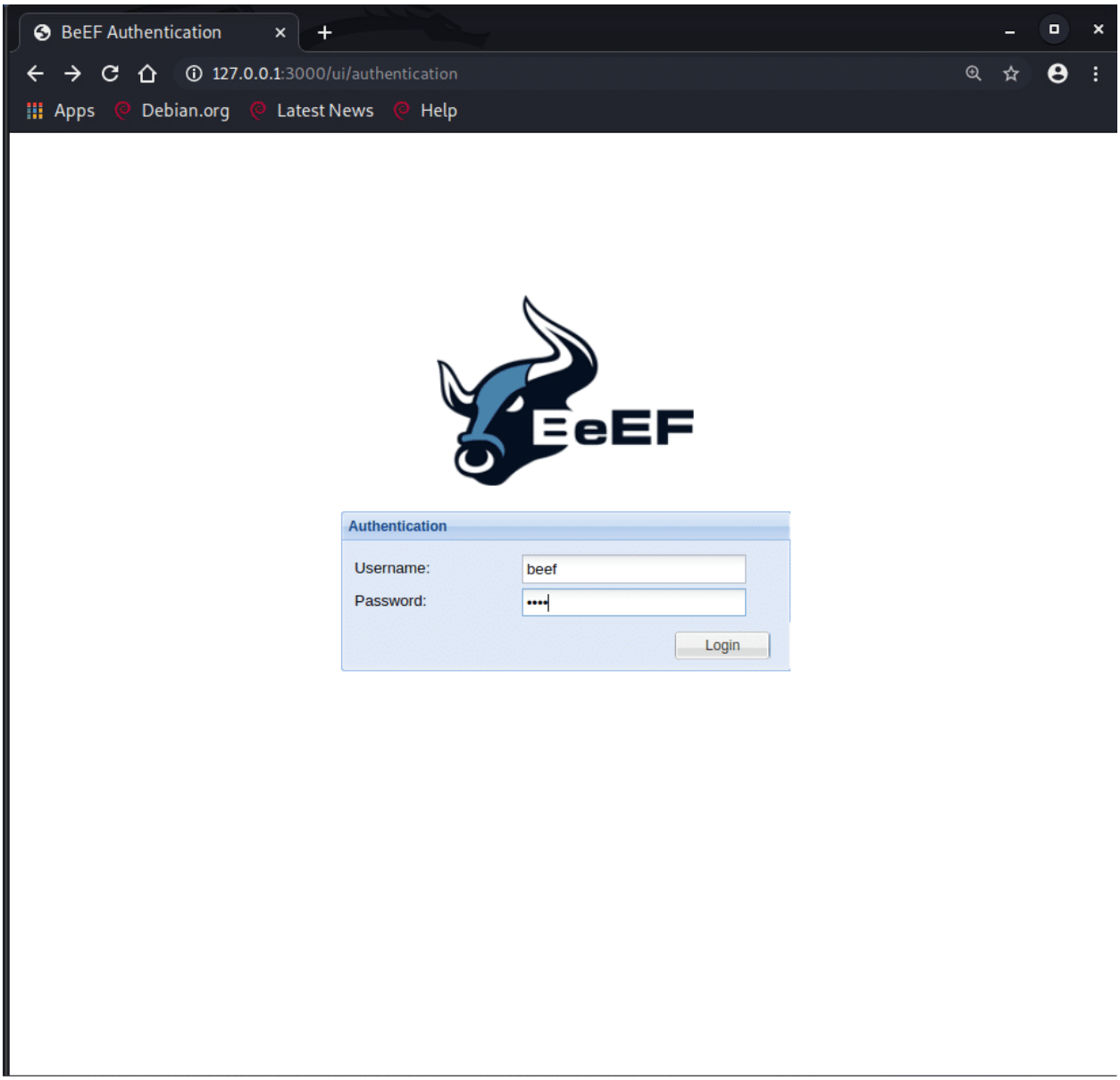

Figure 3: BeEF web UI login page.

And after logging in:

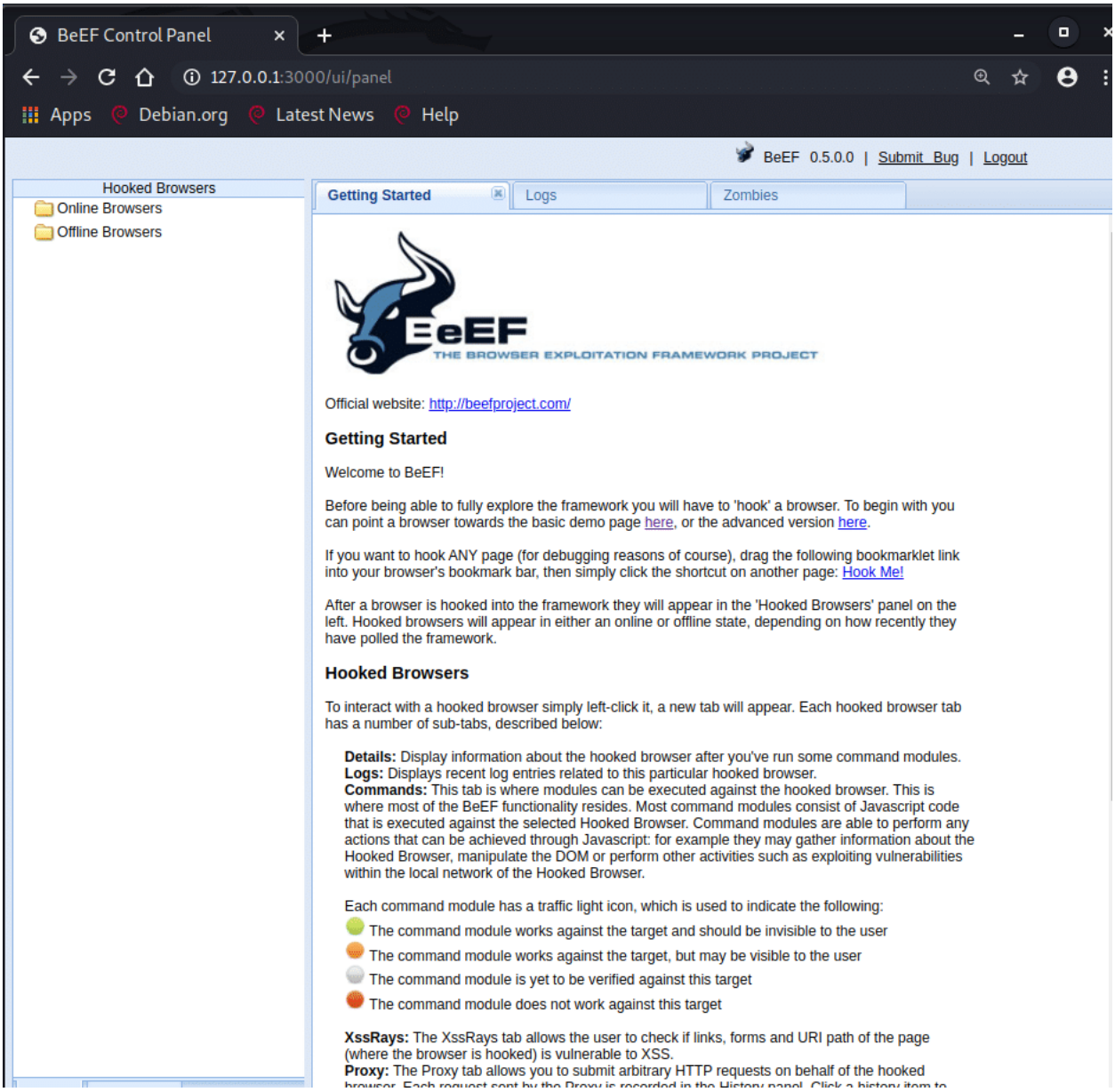

Figure 4: BeEF web UI control panel.

Step 2 – Hooking

Now that the server-side is up and running, we will need to hook our victim's browser by injecting the malicious JavaScript code.

Several attack vectors can be used to inject the code, amongst which are MITM, reflected/stored XSS execution and social engineering. But as JavaScript injection is a complex topic and to keep this demonstration focused, we'll be using BeEF's basic demo page that is already injected with the malicious code.

Here you can see our victim accessing the demo page on his Windows machine through Microsoft Edge:

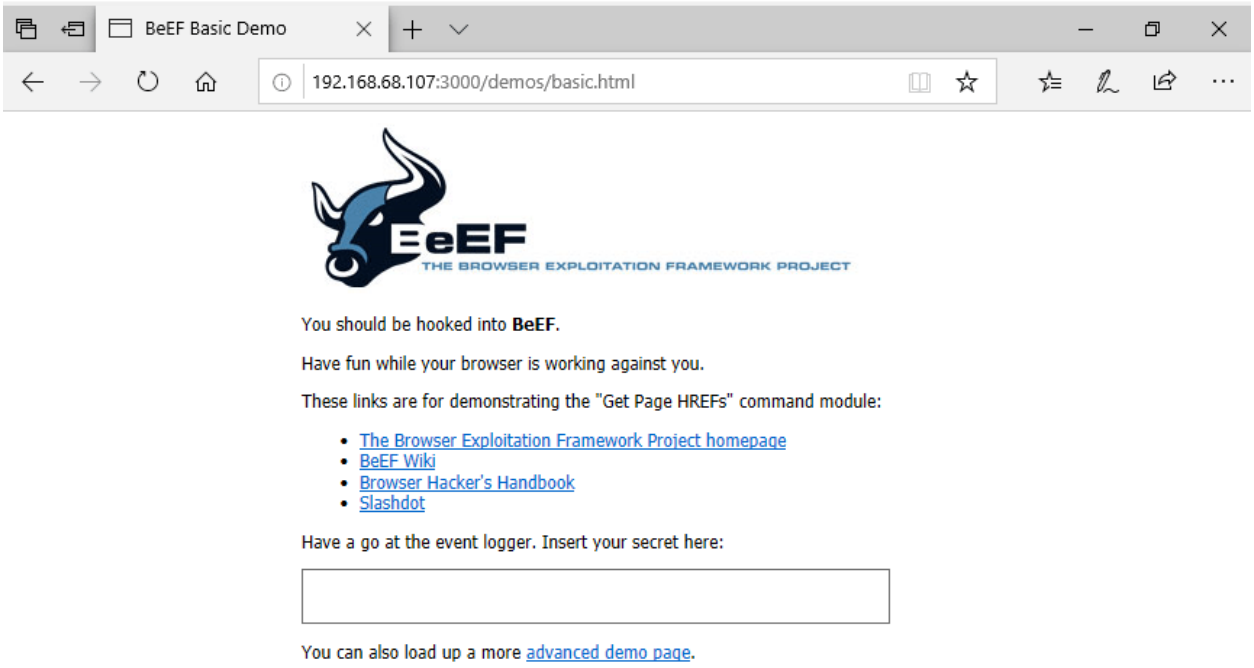

Figure 5: The victims accesses a web page injected with the BeEF hook.

And over on the server side, a new client is shown:

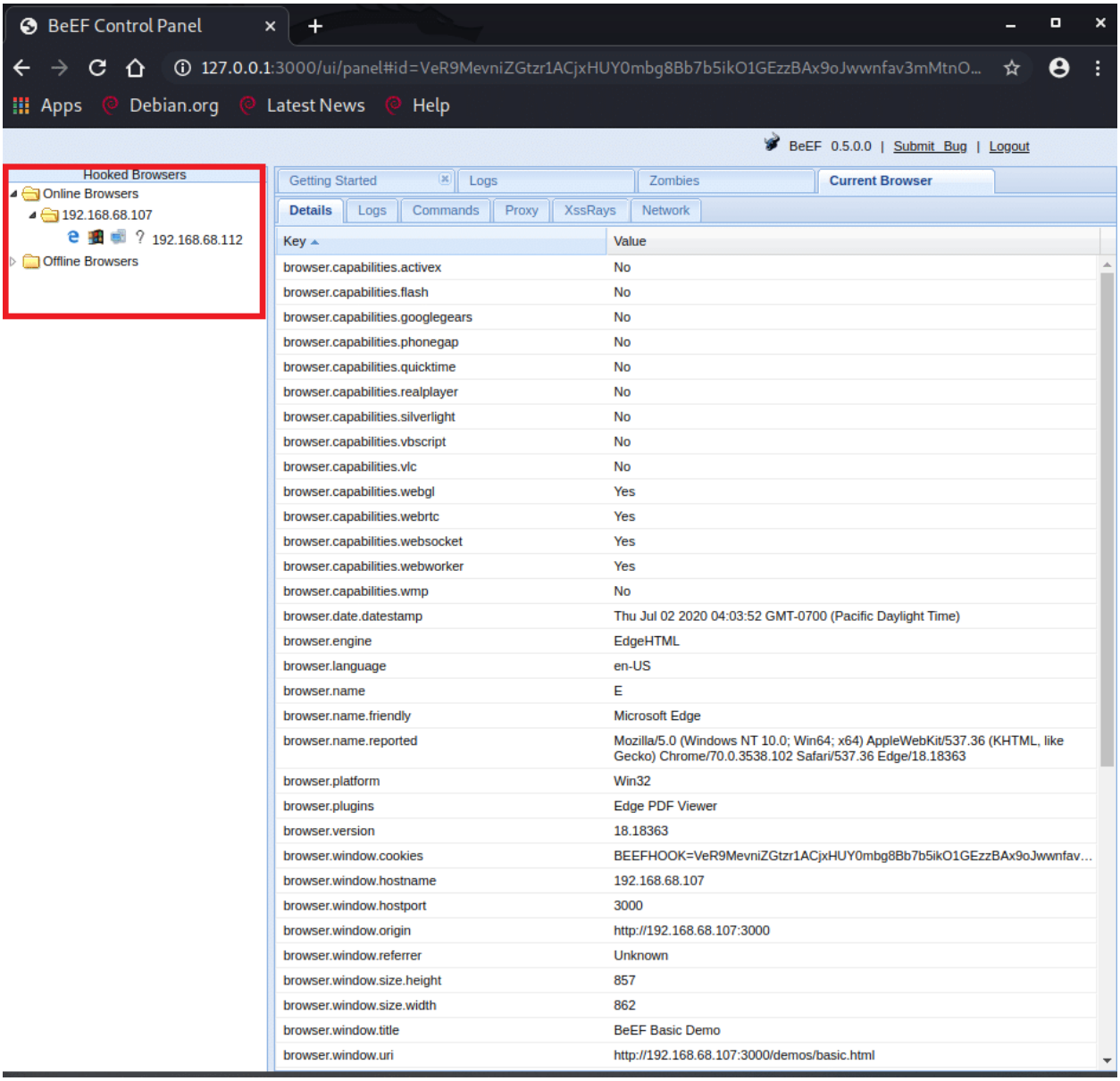

Figure 6: Victim's hooked browser seen in control panel alongisde various details (to the right).

Now that the browser is hooked, we can use BeEF's simple interface and comprehensive list of modules to exploit our victim.

Step 3 – Fingerprinting and Persistence

Alongside the basic data shown on the hooked browser's main screen (see previous figure), many modules are available to fingerprint the victim and establish persistence, among which are:

- Fingerprint Local Network – Discovers LAN devices based on default logo images and favicons for known network devices and applications. IP range and target ports can be customized.
- Fingerprint Browser – Fingerprints browser and browser capabilities using *FingerPrintJS2*.
- Detect Antivirus – Tries to discern installed anti-viruses by detecting JS code that is automatically included by some of them. May detect Kaspersky, Avira, Avast, Norton and others.
- Man-in-the-Browser – Ensures the BeEF hook will persist until the victim manually changes the URL in the URL bar. Originally, the hook is injected into a single web page, so switching web pages will terminate the attack. BeEF handles this by listening for link-clicks and handling the request, instead of letting the browser handle it. Sometimes server policies will prevent this behavior if the link is to a different domain, so in these cases the link will be opened in a new tab to avoid losing the original hook.

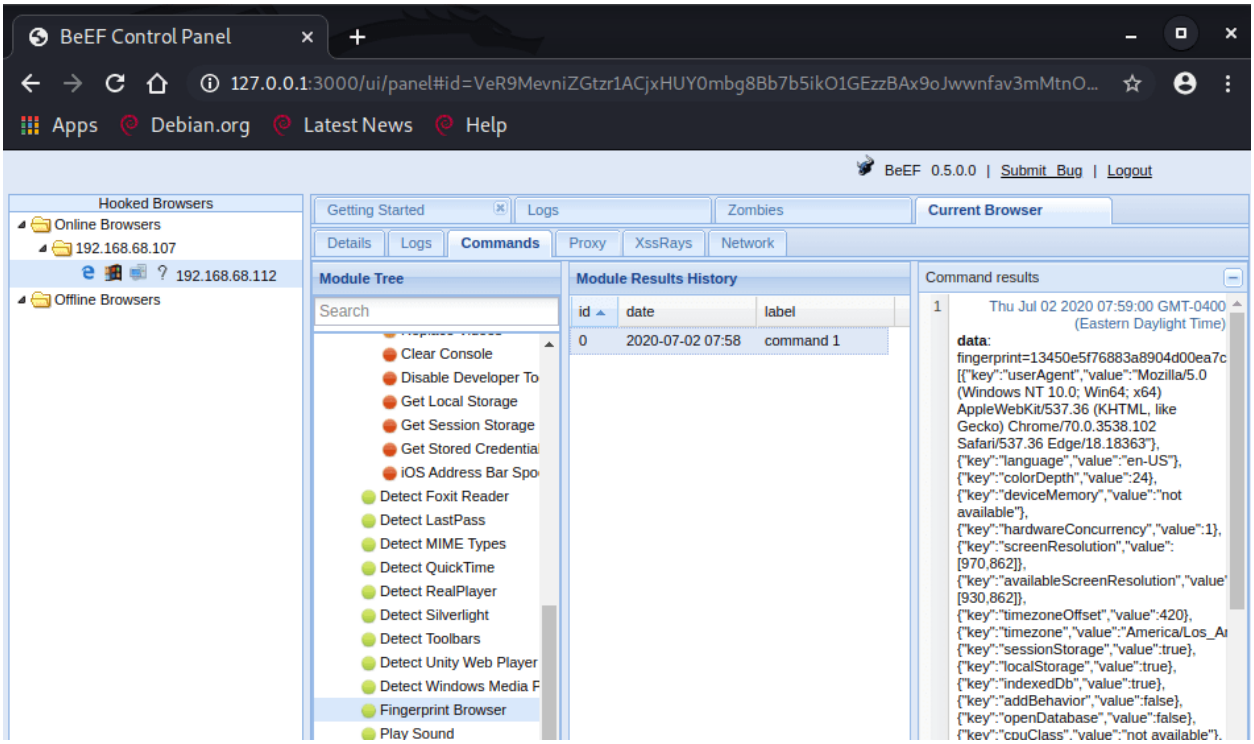

Figure 7: "Fingerprint Browser" command module, with its results in the right-most pane.

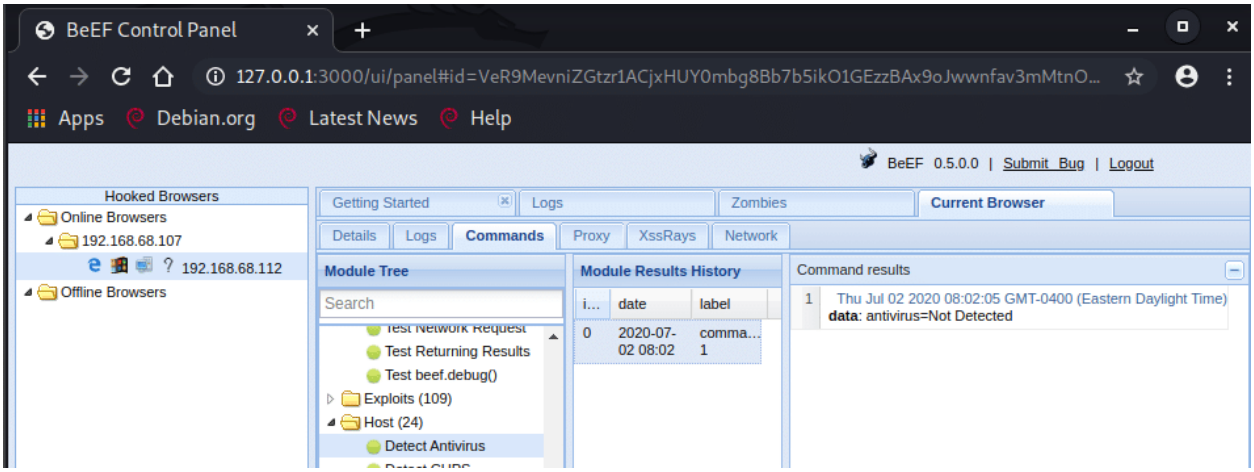

Figure 8: "Detect Antivirus" command module, with its results in the right-most pane.

Step 4 – Data Collection

After assuring persistence and fingerprinting the victim, adversaries may start to collect personal victim data with the following modules (and various others):

- Detect Social Networks – Detects if the hooked browser is currently authenticated to Gmail, Facebook and Twitter.
- Spider Eye – Captures a snapshot of the hooked web page.
- Get Form Values – Retrieves the name, type, and value of all input fields on the page.

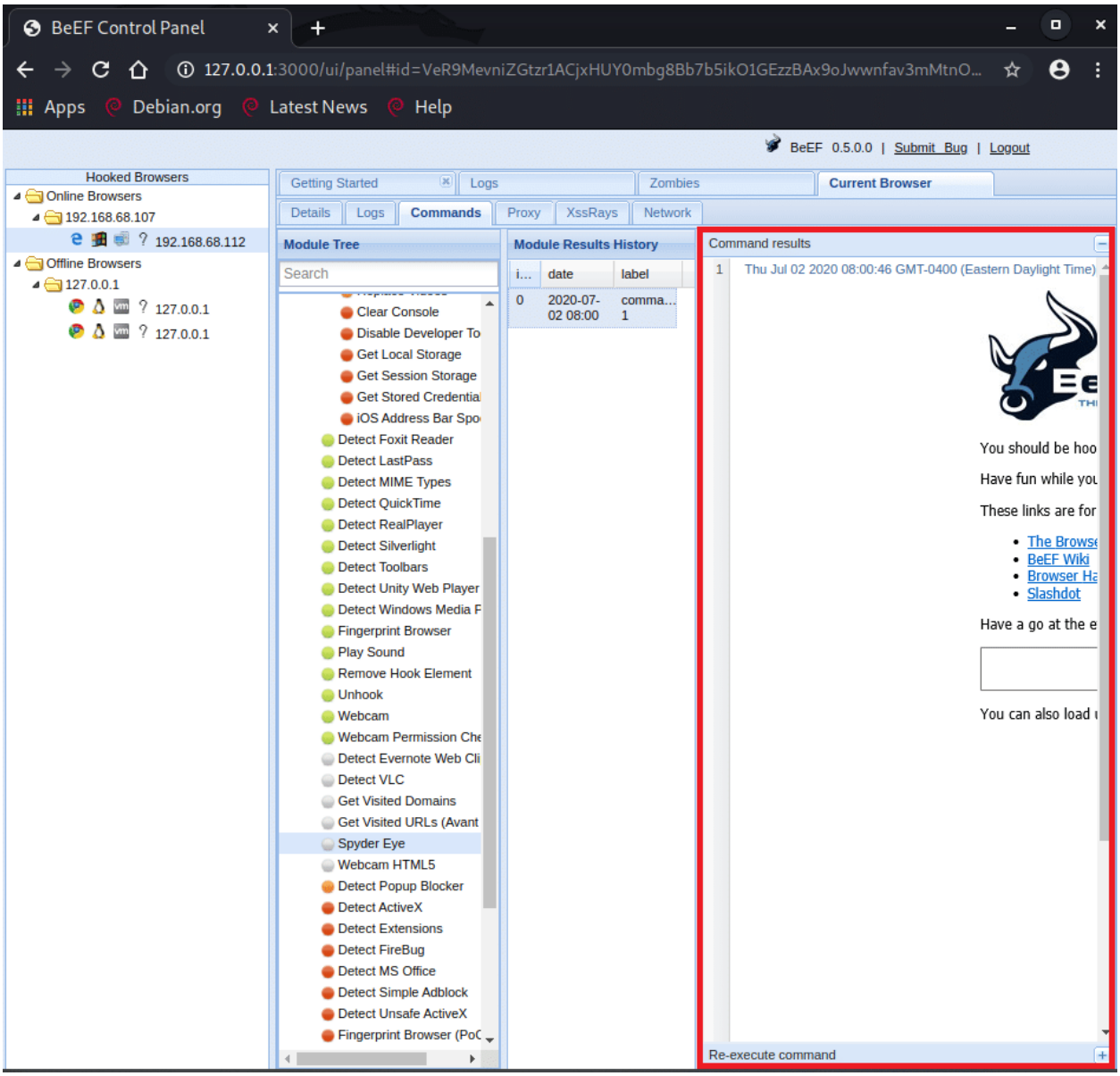

Figure 9: "Spider Eye" module showing the victim's hooked browser tab (marked in red).

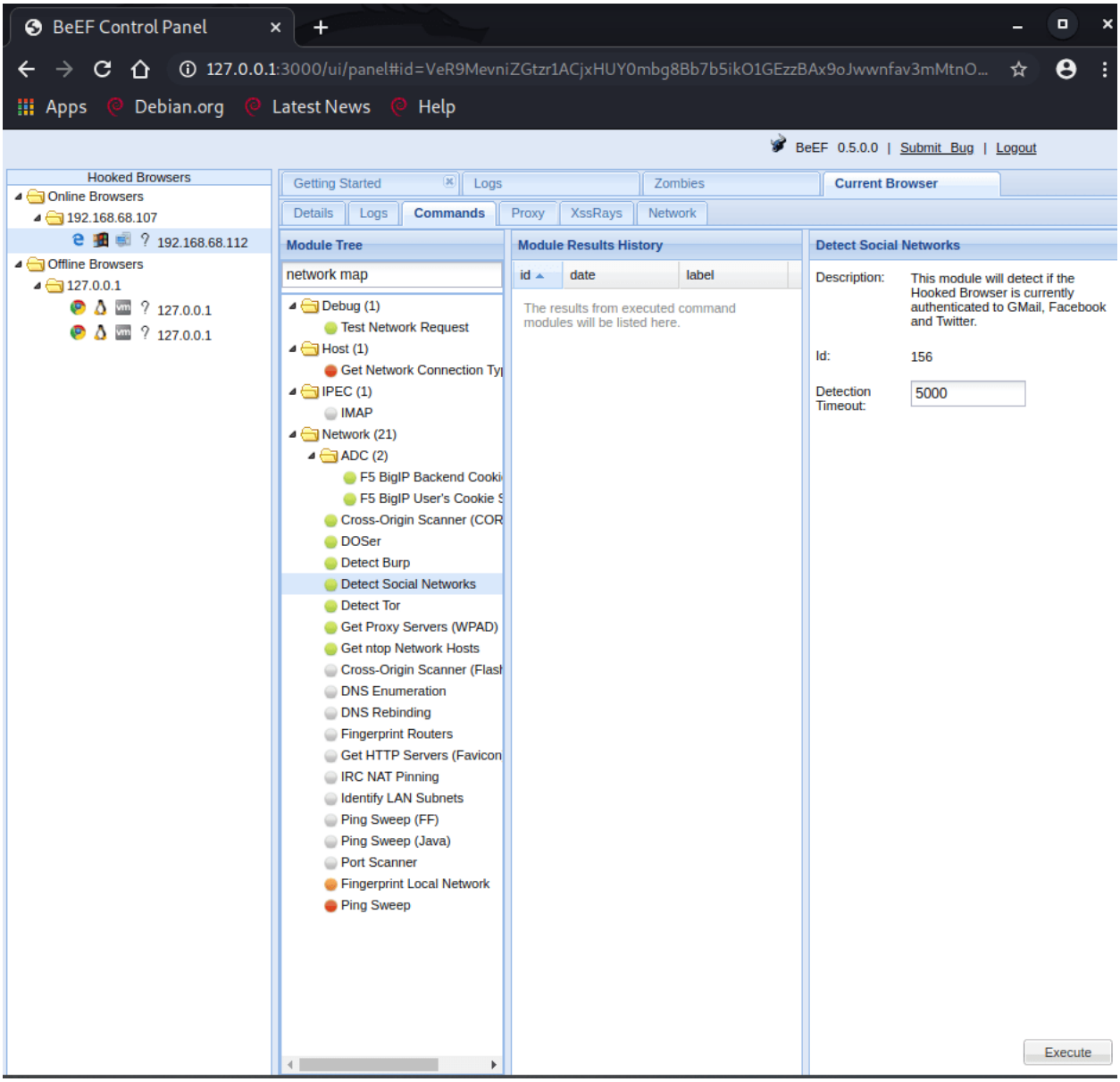

Figure 10: "Detect Social Networks" command module, with its description and parameters in the right-most pane.

Step 5 – Data Manipulation

Adversaries may exploit man-in-the-browser attacks to forge requests on behalf of the victim, in the security context of the hooked browser. The victim "does all the heavy lifting" by authenticating to a certain service, say an Enterprise's cloud storage, and the attacker swoops in at the last moment, sending a request to the storage server to delete all files.

This is accomplished by tunneling the attacker's desired requests to the hooked browser session over BeEF's communication channel, thus mimicking a reverse HTTP proxy. The hooked browser session in the proxy's exit point.

First, the desired hooked session must be set as the proxy server:

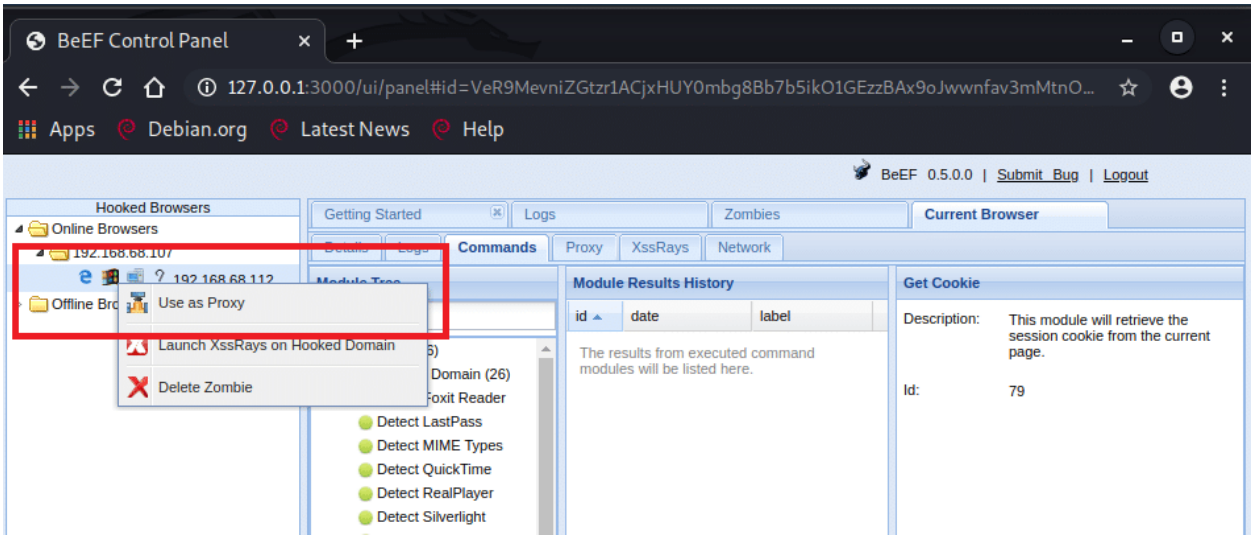

Figure 11: Using the hooked browser as a proxy.

Now requests can be forged through the control interface under *Proxy -> Forge Request*:

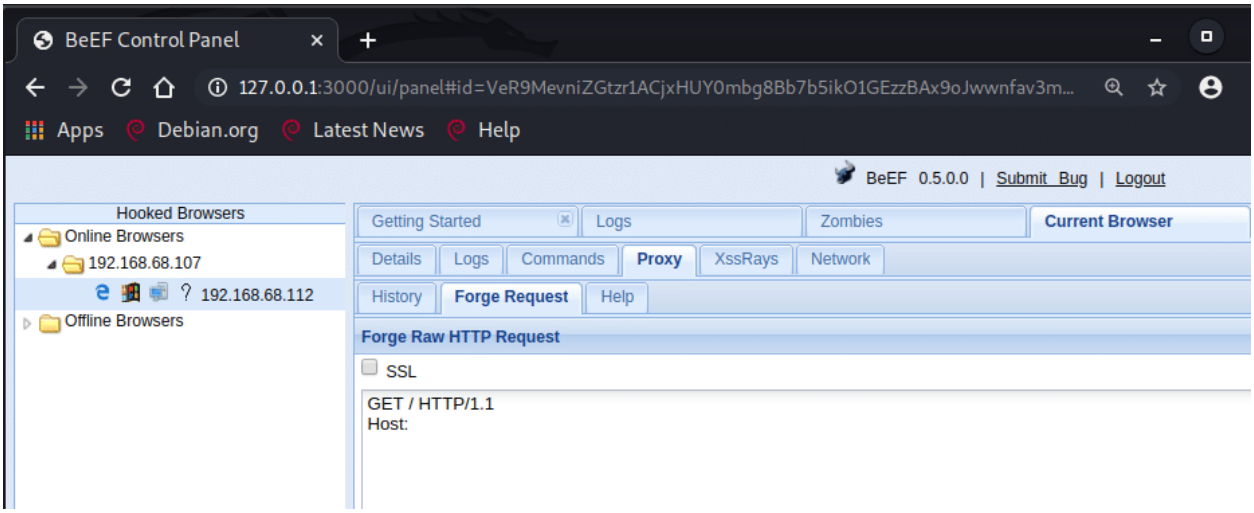

Figure 12: Forging HTTP requests through BeEF's control panel.

Alternatively, a different web browser can be set to use the proxy server, allowing the attacker to visually hijack the victim's browser. To demonstrate this, we set up a dummy website with a simple logon page. Here it is on the victim's

machine:

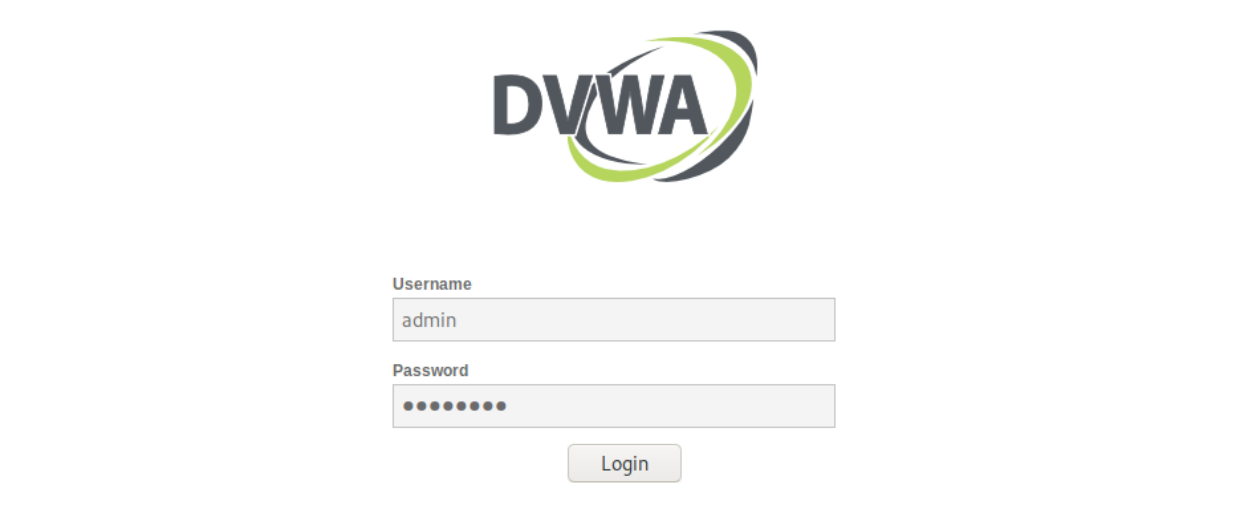

Figure 13: Victim logging in to a secure service.

On the BeEF server, we set up a proxy as shown above, and configured Firefox to use it. Now, when we access the DVWA website, we are not prompted with the login screen – because we're browsing in the victim's security context:

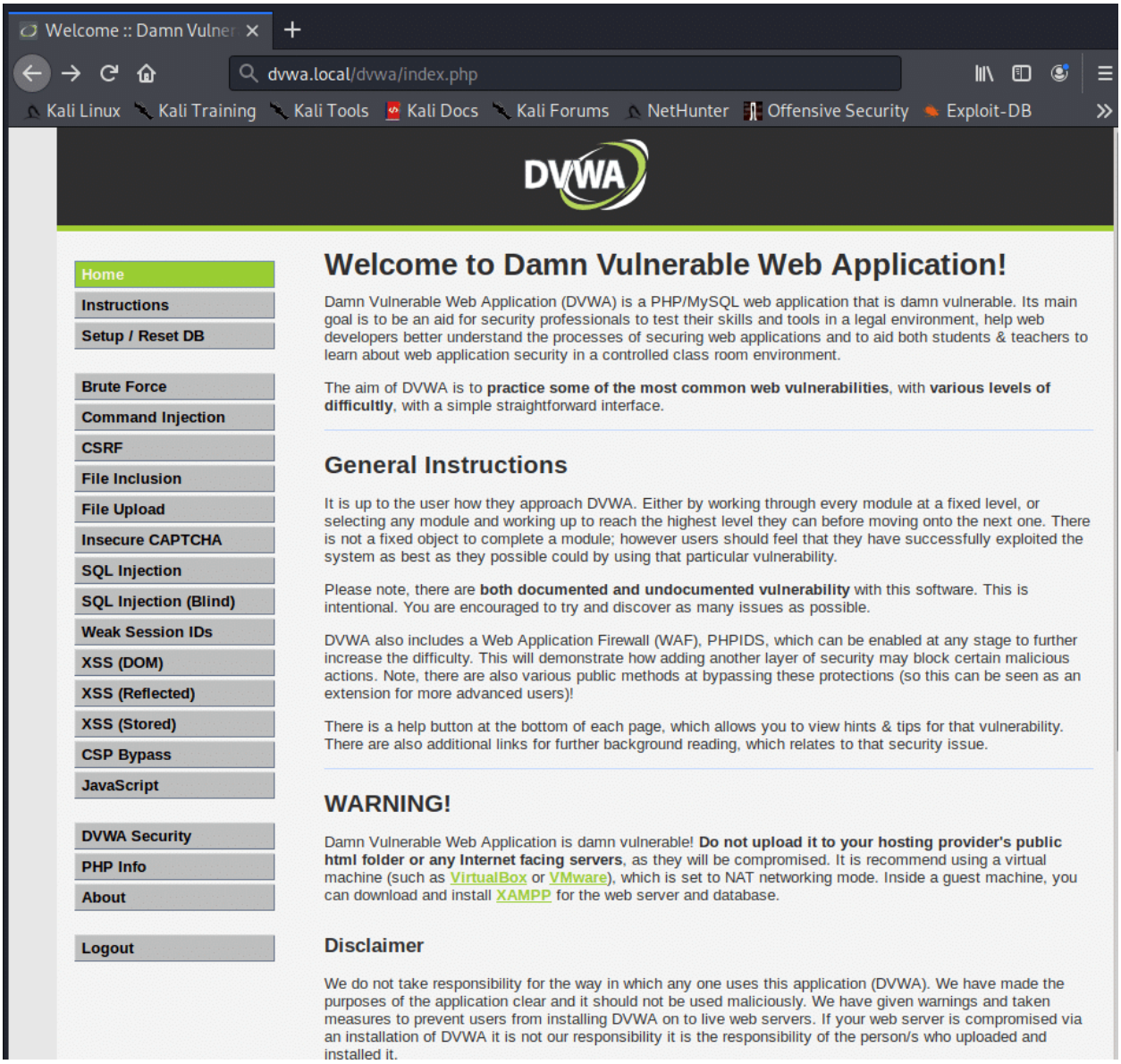

Figure 14: On the attacking machine, the attacker accesses the secure service without authentication, using the hooked browser as a proxy exit point.

Mitigation

Man-in-the-browser attacks are client-based and very hard to detect from a network traffic perspective. Moreover, no new processes are created and sometimes the malicious code will live entirely “off the land”.

Some approaches to detection and prevention may be:

- Out-of-Band Notice/Confirmation (Server-Side) – Expanding on out-of-band authentication, the user is sent a summary of the actions performed in a service. In

certain cases, the notice may even be used as an integral part of the transaction. For example, when transferring funds online, the client may have to confirm the transaction with an out-of-band OTP included in an e-mail with details about the transaction.

- Malware Detection (Client-Side) – as many MITB malwares contain static code files, they may be signed, blocked and removed by antivirus software. Runtime detection techniques can also be used[9].
- User Training – Endpoint users should be trained to identify suspicious browser extensions, close browser sessions regularly when they're no longer needed and avoid social engineering attempts.

Further Reading

Footnotes

1. https://attack.mitre.org/tactics/TA0009/
2. https://attack.mitre.org/techniques/T1115/
3. https://attack.mitre.org/techniques/T1056
4. https://attack.mitre.org/techniques/T1113/
5. https://attack.mitre.org/techniques/T1125/
6. https://attack.mitre.org/techniques/T1185
7. Advances in Decision Sciences, Image Processing, Security and Computer Vision: International Conference on Emerging Trends in Engineering (ICETE), Vol. 1
8. https://www.microsoft.com/en-us/research/project/detection-of-javascript-based-malware/

Man-in-the-Middle & Spoofing

- https://en.wikipedia.org/wiki/Man-in-the-middle_attack
- https://www.cynet.com/attack-techniques-hands-on/llmnr-nbt-ns-poisoning-and-credential-access-using-responder/

Man-in-the-Browser

- https://attack.mitre.org/techniques/T1185/#:~:text=Browser%20traffic%20is%20pivoted%20from,any%20HTTP%20and%20HTTPS%20traffic.

- https://en.wikipedia.org/wiki/Man-in-the-browser/
- https://owasp.org/www-community/attacks/Man-in-the-browser_attack
- https://doubleoctopus.com/security-wiki/threats-and-tools/man-in-the-browser-attack/
- https://en.wikipedia.org/wiki/Browser_security#Plugins_and_extensions